



BUSINESS REQUIREMENTS DOCUMENT

Synthetic Monitoring Platform

Core objectives, scope, and functional requirements.

PREPARED FOR Client Stakeholders • PREPARED BY Agam AI Labs • August 24, 2026

REF. AGAL-2026-SYNTH-001.01.TEX – CLIENT DRAFT



Contents

1	Executive Summary	2
1.1	Purpose of the Document	2
1.2	Business Problem Statement	2
1.3	Proposed Solution	2
2	Project Overview & Objectives	4
2.1	Business Objectives	4
2.2	Success Metrics (KPIs)	4
3	Scope	5
3.1	In-Scope	5
3.2	Out-of-Scope	5
4	User Personas & Roles	6
4.1	System Administrators (The “Configurators”)	6
4.2	Application Owners (The “Responders”)	7
4.3	Executive Stakeholders (The “Reviewers”)	7
5	Functional Requirements (The Core Engine)	8
5.1	Probe & Check Execution Engine	8
5.2	Alerting & Routing Matrix	9
5.3	Unified Service Dashboard	9
5.4	User & Role Management (RBAC)	10
6	Non-Functional Requirements	10
6.1	Performance & Scalability	11
6.2	Security & Compliance	11
6.3	Availability & Reliability	11
7	Dependencies & Assumptions	12
7.1	Third-Party Integrations	12
7.2	Technical Assumptions	13

1 | Executive Summary

1.1 Purpose of the Document

This Business Requirements Document (BRD) defines the core objectives, scope, and functional requirements for a centralized Synthetic Monitoring Platform. It serves as the foundational agreement between the engineering team and the client stakeholders, ensuring alignment on what the system will do, who will use it, and how it will integrate with existing workflows before development begins.

1.2 Business Problem Statement

The client currently operates a fragmented application ecosystem. Services are deployed across a heterogeneous mix of hosting environments, cloud providers, and infrastructure types. This fragmentation creates a critical visibility gap:

ECOSYSTEM CHALLENGES

Critical Visibility Gaps

- **Siloed Monitoring:** Traditional infrastructure monitoring is locked within individual hosting providers, making it impossible to see the global health of the ecosystem.
- **Delayed Detection:** Because there is no outside-in verification, the client often relies on end-users to report downtime or performance degradation.
- **Fragmented Alerting:** When an issue does occur, there is no standardized mechanism to identify the failing application, determine the correct owner, and immediately route an alert to them.

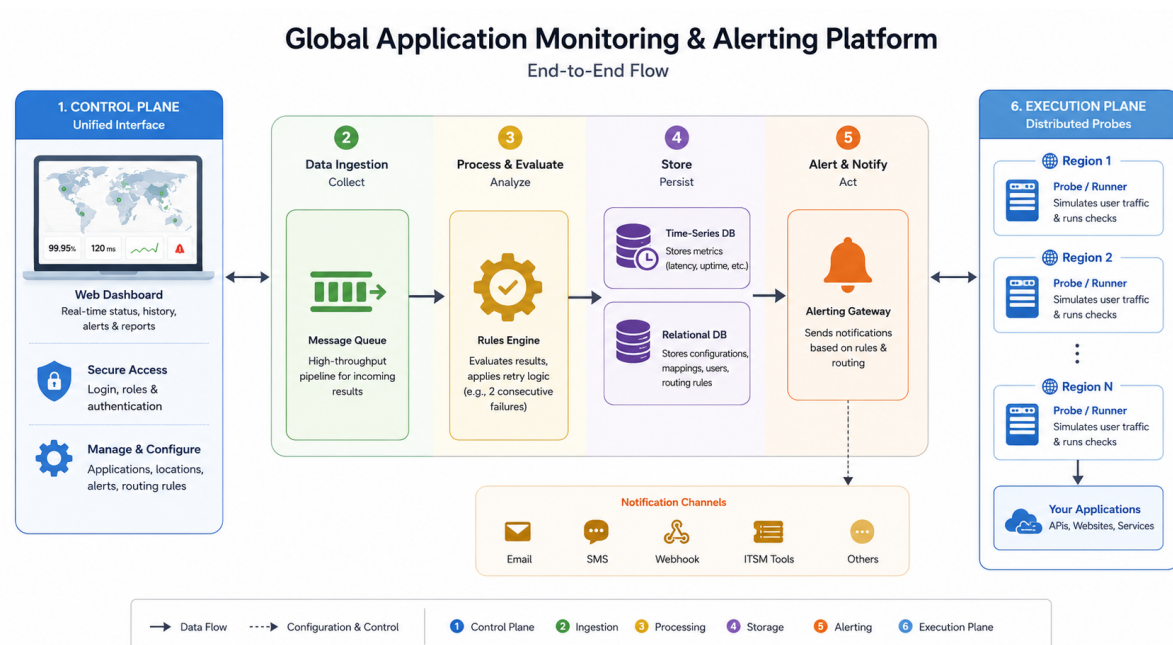
1.3 Proposed Solution

We will build a unified synthetic monitoring application designed to act as the single source of truth for application health, regardless of where those applications are hosted.

STRATEGIC APPROACH

Unified Architecture

- **Active Simulation:** The system will utilize distributed probes to continuously simulate external user traffic (pings, HTTP requests, API calls) against the client's global application roster.
- **Centralized Visibility:** All uptime and latency data will be aggregated into a single, unified dashboard, abstracting away the underlying heterogeneous hosting environments.
- **Intelligent Routing:** Upon detecting a failure, a centralized rules engine will validate the downtime and instantly route automated notifications (via SMS, email, or webhook) directly to the specific application owners, drastically reducing the Mean Time to Detect (MTTD).



2 | Project Overview & Objectives

2.1 Business Objectives

The primary objective is to transition the client from a reactive, decentralized operational model to a proactive, centralized one. Specifically, this platform will:

CORE OBJECTIVES

Strategic Platform Goals

- **Unify Operational Visibility:** Abstract the underlying complexity of the fragmented hosting environments by providing a single, standardized view of application health.
- **Standardize Incident Response:** Eliminate ad-hoc communication by enforcing a centralized routing matrix that automatically identifies and notifies the correct application owner upon failure.
- **Enhance Service Reliability:** Identify downtime and degradation from the end-user's perspective before external customers report the issue.

2.2 Success Metrics (KPIs)

The success of this implementation will be measured against the following targets:

PERFORMANCE TARGETS

Key Performance Indicators

- **Mean Time to Detect (MTTD):** Reduce time to detect application failures to under 5 minutes for tier-1 applications.
- **Proactive Detection Rate:** Ensure >95% of application outages are detected by the synthetic monitoring system before end-users report them via support channels.
- **Alert Delivery Latency:** Route and deliver SMS/Email notifications within 60 seconds of a validated failure.
- **Platform Availability:** The monitoring platform itself must maintain 99.99% up-

time to remain a reliable source of truth.

3 | Scope

3.1 In-Scope

The initial release of the platform will include:

INCLUDED CAPABILITIES

In-Scope Features

- **Outside-In Monitoring:** Execution of synthetic checks including HTTP/HTTPS endpoints, TCP port checks, DNS resolution, and multi-step REST API sequences.
- **Alert Routing Engine:** Configurable retry logic (e.g., “fail twice before alerting”) and a tagging system to map target applications to specific owners.
- **Notification Channels:** Outbound integration for SMS, Email, and generic Webhooks to support standard ITSM tools.
- **Unified Dashboard:** A centralized, web-based UI for real-time status visualization, historical latency reporting, and administrative configuration.
- **Role-Based Access Control (RBAC):** Basic segregation between global administrators and read-only stakeholders.

3.2 Out-of-Scope

To maintain project velocity and system focus, the following are explicitly excluded from this phase:

EXPLICIT EXCLUSIONS

Out-of-Scope Items

- **Application Performance Monitoring (APM):** Internal code profiling, memory leak detection, or database query analysis.
- **Infrastructure Monitoring:** Polling internal server metrics like CPU usage, RAM utilization, or disk space across the heterogeneous hosts.
- **Log Aggregation:** Ingesting and searching application logs (e.g., Splunk/ELK stack functionality).
- **Auto-Remediation:** Automated execution of recovery scripts (e.g., restarting a crashed container or rebooting a server).

4 | User Personas & Roles

This section outlines the primary users interacting with the Synthetic Monitoring Platform, defining their responsibilities, primary goals, and the specific system permissions required to support their workflows.

4.1 System Administrators (The “Configurators”)

PERSONA 01

System Administrator Profile

Profile: IT Operations engineers, SREs, or NOC administrators who are responsible for the overarching health of the monitoring platform itself.

Responsibilities: Adding and deprecating monitored endpoints as the client’s heterogeneous ecosystem evolves; configuring global routing rules; managing integration credentials (e.g., Twilio API keys, SMTP servers); and administering user access.

Primary Needs:

- Bulk configuration capabilities to easily onboard new applications.
- Deep diagnostic tools to troubleshoot why a specific synthetic probe might be failing (e.g., SSL certificate errors vs. DNS resolution failures).
- Full CRUD (Create, Read, Update, Delete) permissions across all system modules.

Viewpoint: Needs a comprehensive, unrestricted view of the entire global matrix, including backend platform health.

4.2 Application Owners (The “Responders”)

PERSONA 02

Application Owner Profile

Profile: Lead developers, product managers, or specific business-unit IT leads who own individual applications hosted within the disparate environments.

Responsibilities: Receiving automated incident alerts, triaging the reported downtime, and orchestrating the internal fix.

Primary Needs:

- Highly accurate alerting with minimal false positives (alert fatigue is their biggest risk).
- Actionable forensic data delivered directly in the notification payload (e.g., “HTTP 502 Bad Gateway at 10:04 AM on AWS us-east-1”).
- Read-only access to the dashboard, specifically filtered to show only the applications assigned to their team.

Viewpoint: Needs a scoped, focused view of their specific assets and historical SLA performance without being distracted by unrelated infrastructure.

4.3 Executive Stakeholders (The “Reviewers”)

PERSONA 03

Executive Stakeholder Profile

Profile: CIOs, CTOs, or IT Directors who need to monitor the aggregate performance of the business's technology investments.

Responsibilities: Evaluating overarching service level agreements (SLAs), managing vendor performance across the different hosting providers, and reporting on business continuity.

Primary Needs:

- High-level, visually intuitive status boards indicating red/yellow/green health across broad categories (e.g., "All E-Commerce Apps").
- Automated roll-up reports showing weekly or monthly uptime percentages (e.g., 99.99%) across the entire ecosystem.
- Zero configuration capabilities to prevent accidental modifications to the monitoring rules.

Viewpoint: Needs macro-level visibility and historical trend data without the technical noise of individual API payloads or trace routes.

5 | Functional Requirements (The Core Engine)

5.1 Probe & Check Execution Engine

The engine must independently simulate and validate user traffic across all internal and external hosting environments.

EXECUTION ENGINE

Simulation & Validation

- **Protocol Support:** The system must natively execute HTTP/HTTPS (GET/POST/PUT/DELETE), TCP port checks, DNS resolution validation, and ping (ICMP).

- **Multi-Step API Sequences:** Must support chaining API calls, extracting tokens from the first response (e.g., OAuth login) and passing them into subsequent requests to validate complex workflows.
- **Execution Parameters:** Administrators must be able to configure check intervals on a per-app basis (e.g., 1-minute, 5-minute, or custom cron schedules).
- **Geographic Distribution:** Probes must be deployable across multiple geographic regions or network zones to validate availability and measure latency from different origin points.

5.2 Alerting & Routing Matrix

The system must eliminate false positives and ensure failures are routed directly to the actionable party.

INCIDENT ROUTING

Alerts & Notifications

- **Validation & Deduplication Logic:** The system must enforce configurable “retry” thresholds (e.g., “require 2 consecutive failures from 2 different geographic locations”) before generating an alert.
- **Tag-Based Routing:** Every application will be assigned metadata tags (e.g., Owner: Billing, Priority: P1). The routing engine must dynamically evaluate these tags against the failure payload to determine the notification destination.
- **Notification Channels:** The engine must push payloads via SMTP (Email), SMS gateways (e.g., Twilio), and standard Webhooks to integrate with ITSM platforms (e.g., ServiceNow, Jira, Slack).
- **Escalation Policies:** If an alert remains unacknowledged for a configurable time-frame (e.g., 15 minutes), the system must automatically escalate the notification to a secondary contact or channel.

5.3 Unified Service Dashboard

The UI serves as the single pane of glass, masking the complexity of the heterogeneous hosting infrastructure.

USER INTERFACE

Single Pane of Glass

- **Global Status Board:** A real-time, color-coded (Red/Yellow/Green) matrix displaying the health of all monitored applications, filterable by host provider, environment, or tag.
- **Forensic Incident Logs:** Clicking into a failed check must reveal the exact error payload, response code (e.g., 502 Bad Gateway), response latency, and SSL certificate status.
- **SLA & Reporting:** The system must automatically calculate uptime percentages and response-time averages over 7, 30, and 90-day periods, exportable as PDF/CSV for executive review.
- **Configuration Interface:** A centralized UI for System Administrators to rapidly create, duplicate, modify, or pause synthetic checks without writing code.

5.4 User & Role Management (RBAC)

ACCESS CONTROL

Security & Roles

- **Role Segregation:** The system must enforce least-privilege access using three distinct tiers: System Administrator (Full CRUD), Application Owner (Read-only + Alert Acknowledgment), and Executive Stakeholder (Read-only dashboards).
- **Single Sign-On (SSO):** Must integrate with the client's existing identity provider (e.g., Okta, Azure AD) via SAML 2.0 or OIDC.

6 | Non-Functional Requirements

6.1 Performance & Scalability

SYSTEM CONSTRAINTS

Performance Targets

- **High Throughput:** The message queue and execution engine must horizontally scale to support a minimum of 10,000 synthetic checks per minute.
- **Low Latency Processing:** Internal processing time (from a probe detecting a failure to the system queuing the alert) must not exceed 500 milliseconds.

6.2 Security & Compliance

COMPLIANCE

Security Standards

- **Data Encryption:** All data at rest must be encrypted using AES-256. All data in transit must utilize TLS 1.2 or higher.
- **Secrets Management:** Authentication credentials used by the probes (e.g., API tokens, basic auth passwords) must be stored in a centralized, encrypted vault and injected only at runtime.
- **Network Isolation:** Internal runner nodes (probes checking private apps behind the firewall) must operate on a strict outbound-only connection model to prevent unauthorized ingress into the client's network.

6.3 Availability & Reliability

RELIABILITY

Uptime & Redundancy

- **High Availability (HA):** The monitoring system's control plane and database tier

must be deployed in an active-active, multi-availability-zone configuration.

- **Uptime Guarantee:** The system itself must achieve an SLA of 99.99%. If a specific probe runner goes offline, the system must automatically redistribute its assigned checks to healthy runners.

7 | Dependencies & Assumptions

7.1 Third-Party Integrations

The successful alerting and incident routing mechanics of the platform rely heavily on the availability and configuration of the following external services:

EXTERNAL DEPENDENCIES

Integration Requirements

- **Notification Gateways:** The client must provide active, funded accounts and API keys for outbound SMS delivery (e.g., Twilio, AWS SNS) and an SMTP relay for email notifications.
- **ITSM/Incident Management:** For webhook integrations to function, the client's internal ticketing or paging systems (e.g., ServiceNow, PagerDuty, Jira Service Management) must be capable of receiving standard JSON payloads and exposing secure endpoint URLs.
- **Identity Management:** The client must provide integration details for an existing SAML 2.0 or OIDC compliant Identity Provider (IdP) to fulfill the SSO requirement.

7.2 Technical Assumptions

SYSTEM ASSUMPTIONS

Technical Prerequisites

- **Endpoint Testability:** We assume the disparate applications across the heterogeneous hosting environments expose valid, reachable endpoints (e.g., /health, standard HTTP landing pages, or dedicated API routes) that can be queried without triggering unintended state changes (like creating duplicate database records).
- **Firewall Egress/Ingress:** For probes deployed inside the client's private network boundaries, we assume the client's network security team will grant the necessary firewall exceptions to allow outbound metric reporting to the central platform queue.
- **Hosting Agnosticism:** We assume the monitoring system does not require agents to be installed on the client's host machines, maintaining the strict outside-in architecture.